The Principles of Diffusion Models

5.1 Flow-Based Models: Normalizing Flows and Neural ODEs

渡邉 有志

November 13, 2026

第 4 章から第 5 章へ

ここまで（第 4 章 Score SDE）

スコア $\nabla_{\boldsymbol{x}} \log p_t$ を学び、
逆時間 SDE ／ PF-ODE の速度場を組み立てて生成する。

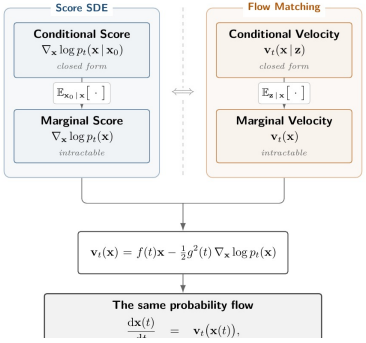
ここから（第 5 章 Flow Matching）

同じ確率流でも、速度場 $\boldsymbol{v}_t$ を**直接**学ぶ道がある。
まず離散の正規化フローから連続の Neural ODE へ進み、
次節で Flow Matching の枠組みに入る。

図5.1：Score SDE と Flow Matching の対応

flow.

A useful mental picture for this chapter is therefore, where both rely on the same conditioning principle:



条件付きの閉じた教師 → 期待で周辺へ。
拡散経路ではスコアから速度が決まり、FM はその速度を直接回帰する。

補足：図 5.1 の読み方

Score SDE

条件付きスコア（閉じた形）
→ 周辺スコア（扱いにくい）
→ PF-ODE 速度へ変換

Flow Matching

条件付き速度（閉じた形）
→ 周辺速度（扱いにくい）
→ 同じ ODE を直接学習

共通の帰結

$\dfrac{d\boldsymbol{x}(t)}{dt} = \boldsymbol{v}_t(\boldsymbol{x}(t))$ で $p_{\text{data}} \leftrightarrow p_{\text{prior}}$ を運ぶ。

5.1 の位置づけ

この節の役割

可逆写像の積み重ね（NF）から連続時間の速度場（NODE）へつなぐ

1. **Normalizing Flows**
   可逆写像・変数変換・層の積み重ね（図 5.2–5.4）
2. **Neural ODEs**
   残差更新の連続極限と尤度の積分形（図 5.5、式 5.1.7–5.1.8）
3. **連続の方程式の直感**
   速度場が密度を運ぶ（連続の方程式への橋）

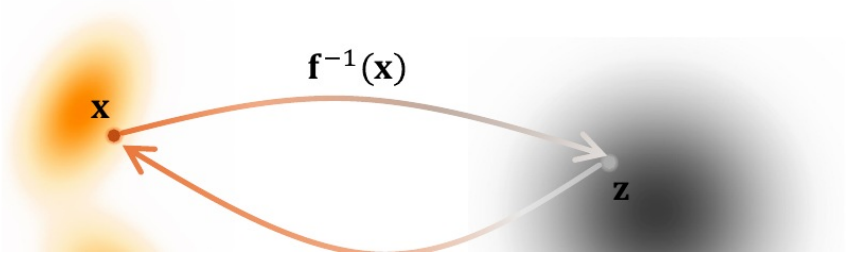
NF ／ NODE に慣れている読者は 5.2 へ進んでもよい（論文の案内）。

区切り：Normalizing Flows

Normalizing Flows

Neural ODEs

図5.2：可逆写像によるサンプル移動



$\boldsymbol{f} : \boldsymbol{z} \mapsto \boldsymbol{x}$（生成）と $\boldsymbol{f}^{-1} : \boldsymbol{x} \mapsto \boldsymbol{z}$（正規化）。
密度の変化は変数変換公式で計算できる。

正規化フローの定義

単純な事前 $p_{\text{prior}}(\boldsymbol{z})$（例：$\mathcal{N}(\mathbf{0}, \boldsymbol{I})$）を、
可逆写像 $\boldsymbol{f}_\phi : \mathbb{R}^D \to \mathbb{R}^D$ で運ぶ：

$$\boldsymbol{x} = \boldsymbol{f}_\phi(\boldsymbol{z}), \qquad \boldsymbol{z} \sim p_{\text{prior}}$$

変数変換より尤度は

$$\log p_\phi(\boldsymbol{x}) = \log p_{\text{prior}}(\boldsymbol{z}) + \log \det \frac{\partial \boldsymbol{f}_\phi^{-1}(\boldsymbol{x})}{\partial \boldsymbol{x}} \tag{5.1.1}$$

学習は最尤 $\mathcal{L}_{\text{NF}}(\phi) = \mathbb{E}_{\boldsymbol{x} \sim p_{\text{data}}}[\log p_\phi(\boldsymbol{x})]$（式 5.1.2）。

図5.3：可逆層の積み重ね

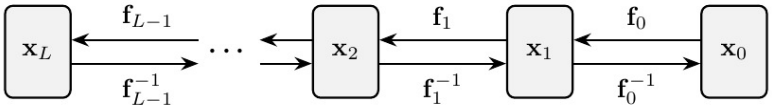


Figure 5.3: Illustration of a NF. An NF consists of a stack of invertible maps $\mathbf{f}_{\boldsymbol{\phi}} = \mathbf{f}_{L-1} \circ \mathbf{f}_{L-2} \circ \cdots \circ \mathbf{f}_0$. The transformation maps latent samples $\mathbf{x}_0 \sim p_{\text{prior}}$ to data samples $\mathbf{x}_L \sim p_{\text{data}}$.

$$\boldsymbol{f}_\phi = \boldsymbol{f}_{L-1} \circ \cdots \circ \boldsymbol{f}_0 \circ$$
$\boldsymbol{x}_0 \sim p_{\text{prior}}$ を $\boldsymbol{x}_L \sim p_{\text{data}}$ へ運ぶ。

積み重ねとログ密度

各層 $\boldsymbol{x}_{k+1} = \boldsymbol{f}_k(\boldsymbol{x}_k)$（式 5.1.3）について

$$\log p_\phi(\boldsymbol{x}) = \log p_{\text{prior}}(\boldsymbol{x}_0) + \sum_{k=0}^{L-1} \log \det \frac{\partial \boldsymbol{f}_k^{-1}}{\partial \boldsymbol{x}_k} \tag{5.1.4}$$

設計のトレードオフ

1 層で複雑な可逆ネット ⇒ ヤコビアンが重い。
単純な可逆層を多く積む ⇒ 各層の det は安く、表現力は積で稼ぐ。

補足：代表的な可逆層

Planar Flow

$\boldsymbol{f}(\boldsymbol{z}) = \boldsymbol{z} + \boldsymbol{u}\, h(\boldsymbol{w}^\top \boldsymbol{z} + b)$
det はスカラー形で安い。

Residual Flow

$\boldsymbol{f}(\boldsymbol{z}) = \boldsymbol{z} + \boldsymbol{v}(\boldsymbol{z})$（式 5.1.5）
収縮なら可逆。
$\log\det$ はトレース展開（5.1.6）。

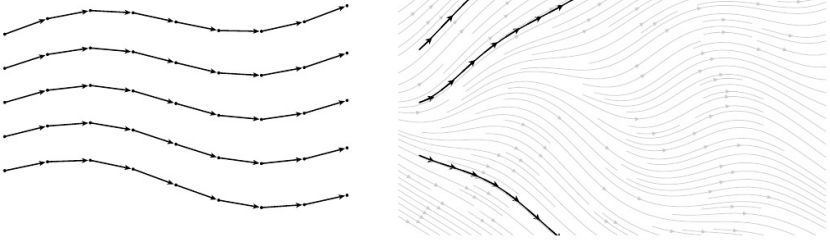
次への橋

残差更新 $\boldsymbol{x} + \boldsymbol{v}(\boldsymbol{x})$ を細かく刻むと、連続時間の ODE に近づく。

図 5.4：離散 NF と連続 NF

$$\mathbf{x}_{k+1} = \mathbf{f}_k(\mathbf{x}_k) : = \mathbf{x}_k + \mathbf{v}_{\phi_k}(\mathbf{x}_k, k)$$

$$\frac{\mathrm{d}\mathbf{x}(t)}{\mathrm{d}t} = \mathbf{v}_{\phi}(\mathbf{x}(t), t)$$



左：有限個の可逆写像（点と矢印）。
右：速度場に沿う積分曲線（Neural ODE）。

NF のまとめ

- 可逆写像で事前 $\leftrightarrow$ データ。尤度は変数変換（5.1.1）（5.1.4）
- 表現力は層の積み重ねで稼ぎ、各層のヤコビアンは軽く保つ
- 残差型の細かい更新は、連続時間の速度場へ自然につながる

次

$\Delta t \to 0$ の極限として Neural ODE を書く。

区切り：Neural ODEs

Normalizing Flows

Neural ODEs

離散層から連続時間へ

各層を小さな残差更新とみなす：

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \Delta t\, \boldsymbol{v}_\phi(\boldsymbol{x}_k, t_k), \qquad t_k = k\Delta t, \quad \Delta t = \frac{T}{L}$$

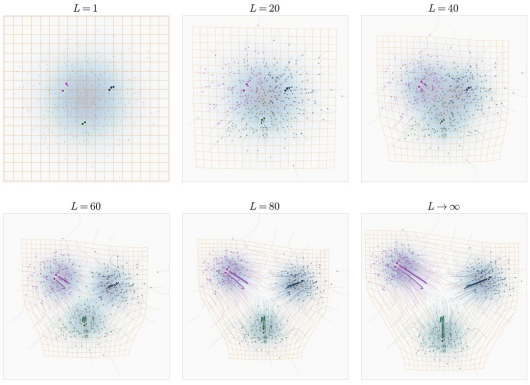
$L \to \infty$、$\Delta t \to 0$ で

$$\frac{d\boldsymbol{x}(t)}{dt} = \boldsymbol{v}_\phi(\boldsymbol{x}(t), t), \qquad t \in [0, T] \tag{5.1.7}$$

呼び名

Neural ODE ／ Continuous Normalizing Flow。
時刻の約束：$t = 0$ が事前、$t = T$（または 1）がデータ。

図 5.5：離散双射から連続輸送へ



L を増やすと、サンプル軌跡と格子変形が滑らかになる。
$L \to \infty$ で時間依存の連続流に近づく。

補足：図 5.5 の読み方

- 色付き点：事前（ガウス）から 3 峰の目標へ運ばれるサンプル
- 橙の格子：空間の変形（体積の伸び縮み）を可視化
- L が小さい：各層の変形が大きい／粗い
- $L \to \infty$：軌跡が滑らかな曲線になり、速度場の積分曲線に見える

密度側

運ばれた密度の時間発展は連続の方程式（式 5.2.8）に従う。

瞬間の変数変換公式

ODE 軌道に沿って

$$\frac{d}{dt}\log p_\phi(\boldsymbol{x}(t),t) = -\nabla_{\boldsymbol{x}}\cdot \boldsymbol{v}_\phi(\boldsymbol{x}(t),t)$$

$t=0\to T$ で積分すると

$$\log p_\phi(\boldsymbol{x}(T),T) = \log p_{\text{prior}}(\boldsymbol{x}(0)) - \int_0^T \nabla_{\boldsymbol{x}}\cdot \boldsymbol{v}_\phi(\boldsymbol{x}(t),t)\,dt \tag{5.1.8}$$

離散との対応

和 $\sum_k \log\det$ が積分 $\int(-\nabla\cdot\boldsymbol{v})\,dt$ に置き換わる。

補足：補題 5.1.1（瞬間の変数変換）

ODE $\dfrac{d\boldsymbol{z}(t)}{dt} = \boldsymbol{F}(\boldsymbol{z}(t), t)$ に従う過程について、
十分正則なら軌道に沿って

$$\frac{d}{dt} \log p(\boldsymbol{z}(t), t) = -\nabla_{\boldsymbol{z}} \cdot \boldsymbol{F}(\boldsymbol{z}(t), t) \tag{5.1.9}$$

見方

連続の方程式＋連鎖律の「軌道版」。
拡散なしの Fokker–Planck（連続の方程式）とつながる。

NODE の学習と推論

学習（最尤）

$\mathcal{L}_{\mathrm{NODE}}(\phi) = \mathbb{E}_{\boldsymbol{x}\sim p_{\mathrm{data}}}[\log p_\phi(\boldsymbol{x}, T)]$。
データ $\boldsymbol{x}$ から逆向きに ODE を解き、発散を積分（式 5.1.10）。

推論（サンプリング）

$\boldsymbol{x}(0) \sim p_{\mathrm{prior}}$ を前向き積分：

$$\boldsymbol{x}(T) = \boldsymbol{x}(0) + \int_0^T \boldsymbol{v}_{\phi^\star}(\boldsymbol{x}(t), t)\, dt$$

実務上の課題

学習ループ内で ODE を繰り返し解く必要があり、計算が重い。

連続の方程式への直感

速度場 $\boldsymbol{v}_t$ が密度 p_t を運ぶとき、質量保存は

$$\partial_t p_t(\boldsymbol{x}) + \nabla \cdot \big(\boldsymbol{v}_t(\boldsymbol{x})\, p_t(\boldsymbol{x})\big) = 0$$

（連続の方程式；論文では式 5.2.8）。

- NODE ／ PF-ODE：粒子は $\dot{\boldsymbol{x}} = \boldsymbol{v}_t(\boldsymbol{x})$ で動く
- 密度側：上式で $\{p_t\}$ が決まる
- 図 5.5 の「格子の変形」は、この輸送の幾何的イメージ

NODE のまとめ

- 離散 NF の残差更新 → 連続 ODE（5.1.7）
- 尤度は発散の時間積分（5.1.8）
- 生成は ODE の数値積分；学習もソルバが要り高い

問い（Question 5.1.1）

学習中に ODE を何度も解かずに、速度場をどう学ぶか？

5.1 全体のまとめ

1. **NF** 可逆写像＋変数変換で尤度評価（5.1.1）（5.1.4）
2. **積み重ね**軽い可逆層の合成で表現力を稼ぐ（図 5.3）
3. **NODE** 連続極限の速度場（5.1.7）と瞬間の変数変換（5.1.8）
4. **橋**スコア $\rightarrow$ 速度ではなく、速度を直接学ぶ動機が立つ

5.1 全体のまとめ：一本の流れ

今日の道筋

可逆写像（NF） → 残差の連続極限（NODE） →「ODE なしで速度を学ぶ」へ

核心

生成＝確率分布を運ぶ速度場の学習。
NODE は尤度積分で学ぶが、シミュレーションがボトルネック。

次へ

5.2 では Flow Matching：条件付き速度への回帰で、
周辺速度をシミュレーションフリーに学ぶ。